

Capture and read bug reports

A bug report contains device logs, stack traces, and other diagnostic information to help you find and fix bugs in your app. To capture a bug report from your device, use the **Take bug report** developer option on the device, the Android Emulator menu, or the `adb bugreport` command on your development machine.

To take a bug report, you must have Developer options (/studio/debug/dev-options) enabled on your device to access the **Take bug report** option.

Capture a bug report from a device

To get a bug report directly from your device, do the following:

1. Enable Developer Options (/studio/debug/dev-options).
2. In **Developer options**, tap **Take bug report**.
3. Select the type of bug report you want and tap **Report**.

After a moment, you get a notification that the bug report is ready, as shown in figure 2.

4. To share the bug report, tap the notification.

Capture a bug report from the Android Emulator

From the Android Emulator, you can use the **File a bug** feature in the extended controls:

1. Click **More**

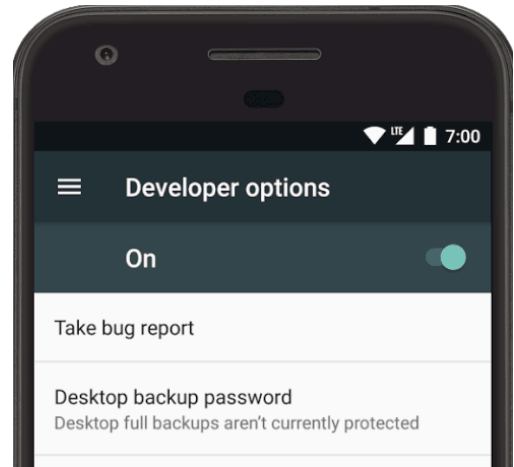


Figure 1. Developer options on a device.

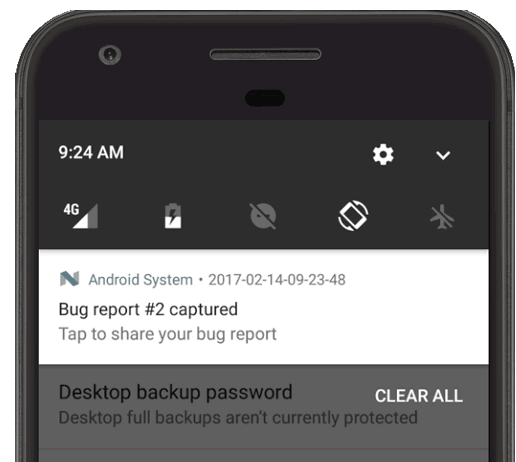


Figure 2. The bug report is ready.

in the Emulator panel.

2. In the **Extended controls** window, select **Bug report**.

This opens a screen where you can see the bug report details, such as the screenshot, the AVD configuration info, and the bug report log. You can also type a message with reproduction steps to save with the report.

3. Wait for the bug report to finish collecting, then click **Save Report**.

Capture a bug report using adb

If you have just one device connected, you can get a bug report using `adb` (/studio/command-line/adb), as follows:

```
$ adb bugreport E:\Reports\MyBugReports
```

If you don't specify a path for the bug report, it is saved to the local directory.

If you have multiple devices connected, you must specify the device with the `-s` option. Run the following `adb` commands to get the device serial number and generate the bug report:

```
$ adb devices
List of devices attached
emulator-5554      device
8XV7N15C31003476  device

$ adb -s 8XV7N15C31003476 bugreport
```

Save an older bug report

By default, bug reports are saved at `/bugreports` and can be viewed using the following command:

```
$ adb shell ls /bugreports/  
bugreport-foo-bar.xxx.YYYY-MM-DD-HH-MM-SS-dumpstate_log-yyy.txt  
bugreport-foo-bar.xxx.YYYY-MM-DD-HH-MM-SS.zip  
dumpstate-stats.txt
```

You can then pull the zip file via `adb pull`:

```
$ adb pull /bugreports/bugreport-foo-bar.xxx.YYYY-MM-DD-HH-MM-SS.zip
```

Inspect the bug report zip file

By default, the zip file is called `bugreport-BUILD_ID-DATE.zip`. The zip file can contain multiple files, but the most important file is `bugreport-BUILD_ID-DATE.txt`. This is the bug report, which contains diagnostic output for system services (`dumpsys`), error logs (`dumpstate`), and system message logs (`logcat`). The system messages include stack traces when the device throws an error and messages written from all apps with the `Log` (`/reference/android/util/Log`) class.

The zip file contains a `version.txt` metadata file that contains the Android release letter. When systrace is enabled, the zip file also contains a `systrace.txt` file. The [Systrace tool](/topic/performance/tracing/command-line) (`/topic/performance/tracing/command-line`) helps analyze the performance of your app by capturing and displaying execution times of your app processes and other Android system processes.

The `dumpstate` tool copies files from the device's file system into the zip file under the `FS` folder so you can reference them. For example, a `/dirA/dirB/fileC` file in the device would generate an `FS/dirA/dirB/fileC` entry in the zip file.

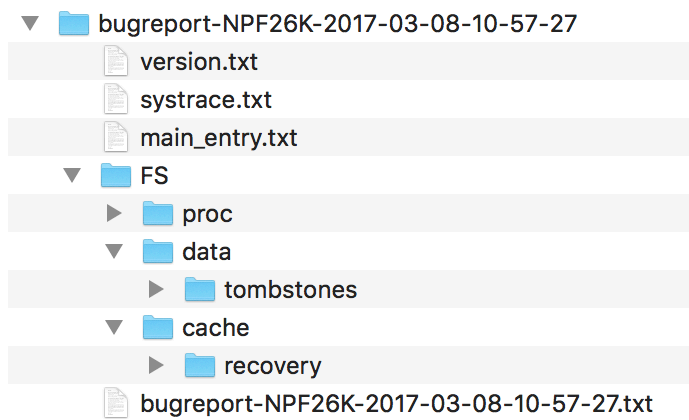


Figure 3. The bug report file structure.

For more information, see [Reading bug reports](https://source.android.com/source/read-bug-reports.html)
(<https://source.android.com/source/read-bug-reports.html>).

Get reports from your users

Capturing bug reports is helpful as you're using the app yourself, but your end-users can't easily share these types of bug reports with you. To get crash reports with stack traces from real-world users, take advantage of the Google Play and Firebase crash reporting features.

Google Play Console

You can get reports from the [Google Play Console](https://play.google.com/console/) (<https://play.google.com/console/>) to view data for crashes and application not responding (ANR) errors from users who installed your app from Google Play. Data is available from the previous six months.

For more information, see [View crashes and application not responding \(ANR\) errors](https://support.google.com/googleplay/android-developer/answer/6083203?ref_topic=7071935) (https://support.google.com/googleplay/android-developer/answer/6083203?ref_topic=7071935) in Play Console help.

Firebase crash reporting

Firebase Crashlytics reporting creates detailed reports of the errors in your app. Errors are grouped into issues based on similar stack traces and triaged by the severity of impact on your users. In addition to automatic reports, you can log custom events to help capture the steps leading to a crash.

To start receiving crash reports from any user, add the Firebase dependencies (<https://firebase.google.com/docs/crashlytics/android/get-started>) to your `build.gradle` file. For more information, see Firebase Crashlytics (<https://firebase.google.com/docs/crashlytics/>).

Content and code samples on this page are subject to the licenses described in the Content License (/license). Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates.

Last updated 2026-03-06 UTC.